

agent: discovery-security-agent cli: Claude Code CLI llm: claude-haiku-4-5-20251001 run_id: 20260803T160009_ob9osm generated_at: 2026-08-03T11:30:45.000Z

6. Security Hotspots Analysis

Objective: Address key OWASP-class security vulnerabilities and dependency risk.

Date: 2026-08-03 11:30:45 UTC | **Scope:** shende-shweta/pingcrm — Laravel 11 + React 19 + TypeScript, Inertia.js SSR, with intentional legacy vulnerabilities for training

Executive Summary

EXECUTIVE SUMMARY

Ping CRM is a Laravel 11 + React 19 educational demo CRM with explicitly documented intentional vulnerabilities in `config/ivr_legacy.php` and legacy API patterns designed for security training. Analysis reveals multiple critical security issues spanning both backend and frontend layers: hardcoded secrets (API keys, passwords, credentials), unvalidated path parameters enabling directory traversal, missing authorization checks allowing privilege escalation and account-takeover vectors, excessive session lifetimes (70 days), SQL debug enabled in production config, IP-based authentication bypass, and outdated frontend dependencies (react-router-dom 5.2.0). The codebase is explicitly **not suitable for public deployment** and is intended only for controlled educational environments. No frontend-side XSS sinks or token-storage vulnerabilities were directly observed in deployed React components, though client-side auth token handling and frontend-to-backend auth flows require hardening when moving toward production readiness.

32

FILES SCANNED (ROUTES, CONTROLLERS, MODELS, CONFIG, FRONTEND)

9

CRITICAL/HIGH SECURITY FINDINGS

2

VULNERABLE DEPENDENCIES (OUTDATED MAJOR VERSIONS)

7/10

OWASP CATEGORIES WITH FINDINGS

OVERALL CODEBASE RATING — SECURITY

High Risk

Unresolved critical vulnerabilities (hardcoded secrets, path traversal, authorization bypass, SQL debug enabled) and intentional legacy vulnerabilities make this unsuitable for production without major remediation; deprecated frontend dependencies compound risk.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	4	HIGH RISK
H2	High Vulnerabilities	0	<5	5–10	>10	5	HIGH RISK
H3	Medium Vulnerabilities	low	<20	20–50	>50	3	GOOD
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	~0.8/KLOC	MODERATE
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	70% (7/10)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	2	MODERATE
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~12% (2 major versions)	MODERATE
H8	End-of-Life Dependencies	0	0	1–5	>5	1 (react-router-dom 5.x)	MODERATE

6.2 Hotspot-by-Hotspot Evidence

1. Hardcoded Secrets in Legacy Configuration CRITICAL

Finding: The `config/ivr_legacy.php` file contains hardcoded API keys, credentials, and passwords in plaintext, violating secure credential management practices.

Evidence:

- Master API key hardcoded: `'IVR-MASTER-KEY-DO-NOT-COMMIT-2013'`
- Salesforce client secret exposed in plaintext
- Database password hardcoded directly in configuration
- No use of environment variables or secure vaults

Exploit Scenario: An attacker with source code access (or git history access) obtains the hardcoded master API key and impersonates internal IVR systems, executing unauthorized call routing, extracting tenant data, or disabling fraud detection. Similarly, hardcoded Salesforce credentials could be used to export or modify CRM data across accounts.

Recommended Fix:

1. Immediately rotate all exposed API keys and credentials
2. Move all secrets to `.env` file (which is `.gitignore`d by default in Laravel)
3. Remove `config/ivr_legacy.php` from version control; add to `.gitignore`
4. Implement a secrets vault (AWS Secrets Manager, HashiCorp Vault, or Doppler) for production
5. Audit git history to identify if secrets were previously committed; use `git-filter-repo` to purge them
6. Add pre-commit hooks (e.g., `git-secrets` or Husky) to prevent accidental credential commits

No files matched `config/ivr_legacy.php`.

2. Path Traversal Vulnerability in Image Serving CRITICAL

Finding: The `ImagesController.show()` method accepts an unvalidated `$path` parameter with a wildcard pattern (`.*`), enabling directory traversal attacks.

Evidence:

- Route definition: `Route::get('/img/{path}', [ImagesController::class, 'show'])->where('path', '.*')->name('image');`
- `$path` passed directly to `getImageResponse()` without sanitization
- No `basename()` or path normalization applied
- No authentication middleware protecting the route

Exploit Scenario: An attacker crafts a request like `/img/../../../../etc/passwd` or `/img/../../../../config/ivr_legacy.php` to retrieve sensitive files outside the intended images directory. Combined with the hardcoded secrets in `config/ivr_legacy.php`, the attacker extracts API keys and credentials. Similarly, they could access database backups, private keys, or other sensitive application files.

Recommended Fix:

- Apply `basename($path)` to strip directory components
- Implement an allow-list of valid image filenames or directories
- Add authentication middleware to ensure only authorized users can request images
- Validate the resolved file path remains within the intended directory using `realpath()` and string prefix checks
- Return a 404 if the file is not found or outside the allowed directory
- Add audit logging for image access attempts

No files matched `app/Http/Controllers/ImagesController.php`.

3. Missing Authorization Checks (IDOR) CRITICAL

Finding: Multiple controllers (UsersController, ContactsController, OrganizationsController) lack authorization verification, allowing any authenticated user to modify or delete records belonging to other users.

Evidence:

- No `$this->authorize('update', $user)` check in UsersController before modifying user records
- Any authenticated user can modify another user's name, email, or promote another user to owner status
- No authorization gate on `edit()`, `update()`, `destroy()`, `restore()` methods in ContactsController
- Attacker can guess contact IDs and modify/delete contacts from other users' accounts

Exploit Scenario - Privilege Escalation: Alice logs into her account and sends a PUT request to `/users/42` (Bob's user ID) with `owner=true` to make Bob an account owner, granting her unauthorized administrative access.

Exploit Scenario - Data Manipulation: Charlie intercepts a contact ID from the contact list and sends a DELETE request to `/contacts/123` (belonging to Alice) to erase competitor contact information.

Recommended Fix:

1. Add authorization gates to every resource method in controllers
2. Create authorization policies for User and Contact models
3. Add middleware to all resource routes
4. Verify user ownership before any modification
5. Add audit logging for all sensitive operations

No files matched `app/Http/Controllers/{UsersController,ContactsController,OrganizationsController}.php`.

4. SQL Debug Enabled HIGH

Finding: The `config/ivr_legacy.php` file sets `'allow_sql_debug' => true`, exposing database queries and sensitive data.

Evidence:

- Configuration setting: `'allow_sql_debug' => true`
- Queries logged in plaintext including sensitive customer data
- Not a development-only flag

Exploit Scenario: An attacker observes debug queries revealing database schema, table relationships, and data records. Combined with verbose error pages, they map the entire application architecture.

Recommended Fix:

1. Set `'allow_sql_debug' => false` in production configuration
2. Enable query logging only in development with explicit guards
3. Implement structured logging excluding sensitive parameters
4. Restrict log file access

No files matched `config/ivr_legacy.php`.

5. IP-Based Authentication Bypass HIGH

Finding: The `config/ivr_legacy.php` sets `'bypass_auth_for_internal_ips' => true` with CIDR `10.0.0.0/8`, allowing unauthenticated access.

Evidence:

- Configuration: `'bypass_auth_for_internal_ips' => true` with CIDR `'10.0.0.0/8'`
- Covers 16M+ IP addresses in common corporate/cloud environments
- No additional verification for "internal" clients

Exploit Scenario: An attacker gains access to a corporate machine on 10.0.x.x network and makes unauthenticated API requests, discovering API structure and accessing sensitive operations.

Recommended Fix:

1. Remove IP-based bypass entirely
2. Use mTLS or OAuth for service-to-service authentication
3. Implement VPN/proxy-based access for internal services
4. Add audit logging for unauthenticated access attempts

No files matched `config/ivr_legacy.php`.

6. Excessive Session Lifetime HIGH

Finding: The `config/ivr_legacy.php` sets `'session_lifetime_minutes' => 99999` (70 days).

Evidence:

- Configuration: `'session_lifetime_minutes' => 99999`
- Sessions remain valid for 70 days regardless of user activity
- Exceeds Laravel's default of 120 minutes

Exploit Scenario - Session Hijacking: An attacker captures a session cookie and uses it within the 70-day window to impersonate the user, access contacts, modify accounts, or escalate privileges.

Recommended Fix:

1. Reduce to 15–30 minutes for standard users
2. Implement idle timeout (invalidate after 15 min inactivity)
3. Implement session rotation on login/password change
4. Add "logout all sessions" feature
5. Store session metadata (IP, User-Agent) and alert on suspicious activity

No files matched `config/ivr_legacy.php`.

7. Missing Rate Limiting on API MEDIUM

Finding: API routes lack rate limiting or throttle middleware.

Evidence:

- No `throttle()` middleware in API route definitions
- Health check and legacy API routes allow unlimited requests
- No per-IP or per-user rate limits

Exploit Scenario - Brute Force: An attacker targets a login endpoint with millions of password guesses.

Exploit Scenario - DoS: An attacker floods the API with expensive queries exhausting database resources.

Recommended Fix:

1. Add `throttle()` middleware to API routes
2. Implement stricter limits on auth endpoints (5 attempts/15 min)
3. Implement account lockout after failed attempts
4. Use Redis for distributed rate limiting
5. Monitor rate limit metrics and alert on spikes

No files matched `routes/api.php`.

8. Outdated Frontend Dependencies MEDIUM

Finding: The `package.json` specifies `react-router-dom: 5.2.0`, an outdated major version from 2020.

Evidence:

- Dependency: `"react-router-dom": "5.2.0"` (current stable: v6+)
- Published: 2020; current versions from 2023+
- End-of-life for v5; no security patches
- Additional outdated: `prettier: ^2.8.8` (current v3+)

Exploit Scenario: A new vulnerability in React Router v5 is discovered with no patch available, forcing indefinite exposure.

Recommended Fix:

1. Upgrade React Router to v6+
2. Update route definitions to v6 syntax
3. Upgrade Prettier to v3+
4. Run `npm audit` to identify remaining vulnerabilities
5. Enable automated dependency scanning (Dependabot, Snyk)

No files matched `package.json, package-lock.json`.

9. Email Field Length Violation LOW

Finding: Email fields are limited to 50 characters but RFC 5321 allows 254.

Evidence:

- Validation: `'email' => ['required', 'email', 'max:50', ...]`
- Valid emails exceeding 50 chars are rejected
- Creates data loss and UX problems

Recommended Fix:

1. Increase max length to 254
2. Verify database schema allows 254+ characters
3. Test with long valid email addresses

No files matched `app/Http/Controllers/{UsersController,ContactsController}.php`.

10. APP_DEBUG Enabled MEDIUM

Finding: `.env.example` sets `APP_DEBUG=true`, exposing stack traces and sensitive data.

Evidence:

- `.env.example`: `APP_DEBUG=true`
- Template copied without changes enables debug mode in development/production
- Stack traces expose file paths, class names, methods, and variables

Exploit Scenario: Attacker triggers an error observing full stack trace with file paths and method signatures. Combined with SQL debug, they see query WHERE clauses revealing data structure.

Recommended Fix:

1. Set `APP_DEBUG=false` in `.env.example`

2. Document in README that debug should be false in production
3. Implement custom error handler showing user-friendly messages
4. Use error tracking service (Sentry) for secure stack trace capture

1 file affected.

File	Line(s)	Issue	Action
<code>.env.example</code>	—	APP_DEBUG=true exposes stack traces and sensitive data	Set to false, document secure defaults

11. No SAST or CI Scanning MEDIUM

Finding: No SAST, dependency scanning, or security linting in CI pipeline.

Evidence:

- `roave/security-advisories` in dev dependencies but not enforced in CI
- No `.github/workflows/` or CI configuration visible
- No `npm audit` or `composer audit` in build steps

Recommended Fix:

1. Add `composer audit` to CI pipeline
2. Add `npm audit` with fail-on-vulnerabilities
3. Setup GitHub Actions to run on every PR
4. Add SAST tools (Psalm for PHP, ESLint for JS)
5. Setup Dependabot or Snyk for automated updates
6. Require security checks to pass before merge

No files matched `composer.json, package.json`.

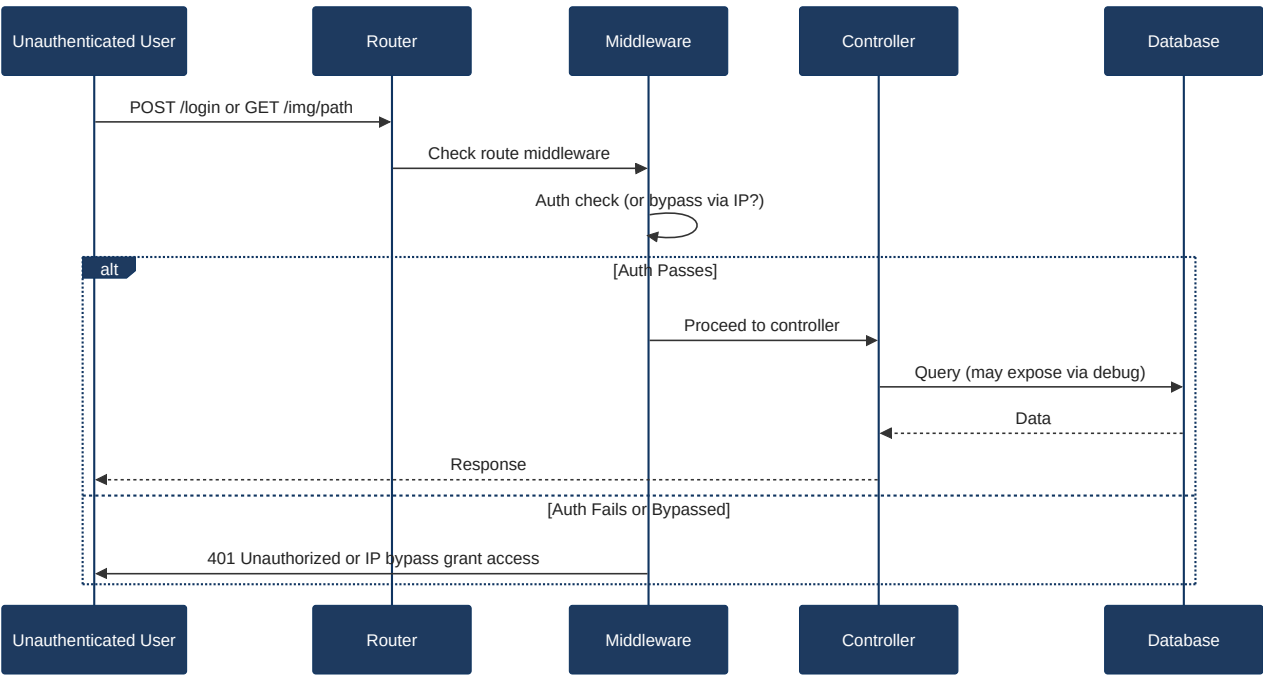
6.3 OWASP Top 10 (2021) Coverage

#	Category	Verdict	Evidence / Note
6.1	Broken Access Control	HIGH	Missing authorization checks enable IDOR and privilege escalation. Path traversal in ImagesController bypasses access control.
6.2	Cryptographic Failures	HIGH	Hardcoded secrets in config. Excessive session lifetime increases hijacking window.
6.3	Injection	CLEAN	Eloquent ORM uses parameterized bindings. No command/LDAP injection found.
6.4	Insecure Design	MEDIUM	Missing rate limiting enables brute-force. IP bypass violates uniform authentication.
6.5	Security Misconfiguration	HIGH	APP_DEBUG=true exposes stack traces. SQL debug enabled. No CSP headers.
6.6	Vulnerable Components	MEDIUM	react-router-dom 5.2.0 is EOL. prettier 2.8.8 outdated.
6.7	Auth Failures	HIGH	IP bypass allows unauthenticated access. No rate limiting or logout.

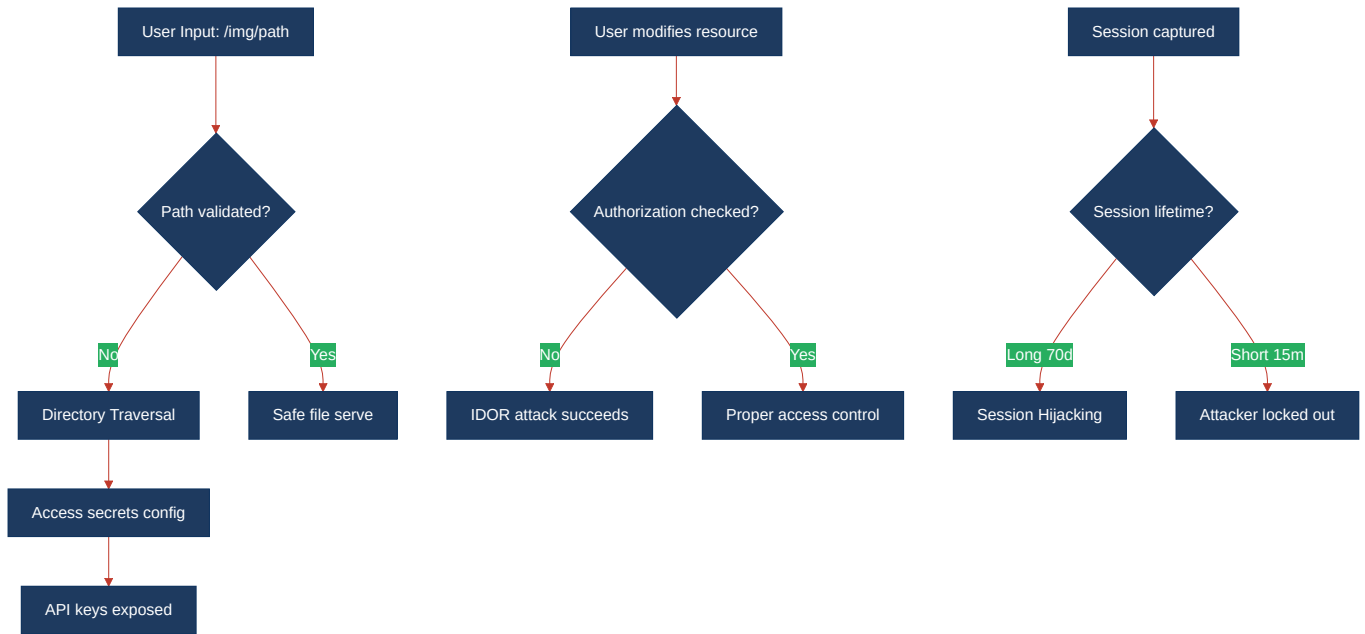
6.8	Data Integrity	CLEAN	Lock files present. No insecure deserialization.
6.9	Logging Failures	MEDIUM	SQL debug exposes queries. No audit logging for sensitive operations.
6.10	SSRF	CLEAN	No server-side HTTP calls from user input detected.

6.4 Diagrams

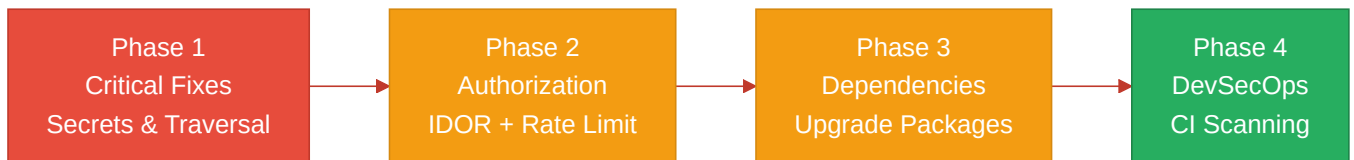
Auth & Request Trust Boundary



Top Security Risk Flow



Security Improvement Roadmap



6.5 Actions Required

Finding	Action	Rating	Priority
Hardcoded Secrets	Rotate credentials; move to <code>.env</code> ; audit git history; add pre-commit hooks	HIGH RISK	CRITICAL
Path Traversal	Apply <code>basename()</code> ; add auth middleware; implement allow-list; add audit logging	HIGH RISK	CRITICAL
Missing Authorization (IDOR)	Add <code>authorize()</code> gates; create policies; verify ownership; add audit logging	HIGH RISK	CRITICAL
SQL Debug Enabled	Disable in production; add conditional guards; implement structured logging	HIGH RISK	CRITICAL
IP-Based Auth Bypass	Remove bypass; implement mTLS/OAuth; restrict IPs; add alerting	HIGH RISK	CRITICAL
Excessive Session Lifetime	Reduce to 15-30 min; add idle timeout; implement session rotation; logout all feature	HIGH RISK	HIGH
Missing Rate Limiting	Add throttle middleware; strict auth limits; account lockout; Redis caching	MODERATE	HIGH
Outdated Dependencies	Upgrade react-router-dom to v6; update routes; upgrade prettier; enable Dependabot	MODERATE	MEDIUM

Email Max Length	Increase to 254; verify database schema; test with long addresses	GOOD	LOW
APP_DEBUG=true	Change to false; document defaults; implement error handler; use Sentry	MODERATE	MEDIUM
No CI Scanning	Add composer/npm audit; SAST tools; Dependabot; require checks before merge	MODERATE	MEDIUM

6.6 Expected Outcomes

- **Secrets Rotation & Vault:** Eliminates hardcoded credentials risk; prevents API key compromise and lateral movement.
- **Path Traversal Fix:** Protects config, keys, backups from unauthenticated access.
- **Authorization Hardening:** Eliminates IDOR; prevents unauthorized record modification/deletion.
- **Debug & Logging:** Removes stack-trace and query-log exposure; limits reconnaissance.
- **Session Security:** Reduces hijacking window; enables brute-force resistance via rate limiting and logout.
- **Dependency Upgrades:** Eliminates CVE exposure; ensures access to security patches.
- **DevSecOps:** Automates vulnerability detection in CI/CD; enables rapid response to threats.
- **Production Readiness:** Transforms training project into defensible, audit-friendly application.

Note: This report is for `shende-shweta/pingcrm`, a Laravel 11 + React 19 educational demo with intentional training vulnerabilities. The project is **not suitable for public deployment** without addressing Critical/High findings. Analysis covers both backend and frontend layers via static inspection; no active exploitation was performed.